

1 Project Description

In this project, students will apply the concepts and theories they have learned in the course to develop an intelligent system that can perform language identification. Through this project, students should be able to demonstrate the following learning outcomes:

- **LO3.** Collaboratively design and implement machine learning models for supervised and unsupervised tasks
- **LO5.** Articulate ideas and present results in correct technical written and oral English.

2 Project Specifications

This section contains the specifications for this major course output.

2.1 Overview

In this project, you will train and evaluate a supervised machine learning classifier, specifically a word-level **language identifier** tailored for Filipino code-switched text. Filipino is a morphologically rich language, featuring complex word formations through affixes, infixes, and reduplication, which can make automated processing challenging. Additionally, Filipino exhibits code-switching, a common linguistic phenomenon where speakers alternate between languages within a single utterance or text. This can occur at the inter-word level (e.g., mixing full words from different languages, such as “bawal corrupt”) or intra-word level (e.g., applying Filipino morphological rules to English stems, such as “pinull” or “naglunch”).

The goal of this project is to build an intelligent system that can accurately identify the language of each word in a code-switched text. Language identification is an important role in the development natural language processing (NLP) applications such as machine translation, sentiment analysis, and speech recognition. This is especially true in multilingual contexts like the Philippines where English and Filipino are frequently intermixed. By addressing training intelligent systems that are sensitive to code-switching, your system can contribute to the better handling of real-world. Filipino data.

2.2 Data Annotation

The foundation of your language identifier will be a high-quality dataset. The dataset will be provided by the Center for Language Technologies (CeLT) of the College of Computer Studies, sourced from the Corpus of Historical Filipino English (or CoHF_{FiE}). This corpus includes historical texts that exhibit code-switching between Filipino and English, reflecting real-world usage in diverse contexts.

However, the dataset that will be provided is **unlabelled**. Therefore, the first step of this major course output for the students to provide labels for each word in the dataset. The labels can either be: **ENG** for English, **FIL** for Filipino (for the purposes of this project, we will consider Tagalog as Filipino as well, even though technically they are distinct), **CS** for intra-code switched words, and **OTH** for other. Please follow the instructions of your instructor regarding how the annotation process should be conducted, as it will differ depending on your section.

While it may sound tedious, it is essential to go through this manual validation process to (1) appreciate the importance of the role of data collection and annotation in the machine learning process, (2) help you understand the nuances of Filipino morphology and code-switching, (3) and foster a culture of collaboration in further advancing the state of NLP in the Philippines. Please **do NOT** use generative AI to annotate the dataset, as these tools are not perfect and may result in low-quality annotations. You can use generative AI only as a guide, but please ensure that all final annotations are done by the group members themselves, and that they are carefully validated.

2.2.1 Criteria for Annotation

In validating the annotated dataset, please **read the following guidelines carefully and ensure compliance**.

Each word must be tagged with the following labels, whenever appropriate.

Tag	Definition
FIL	The word is purely Filipino or Tagalog in origin and form, including those with native affixes (e.g., “bawal”). This also includes loanwords that are presented in Filipino spelling (e.g. “keyk”).
ENG	The word is an English word and uses an English spelling, even if used in a Filipino context (e.g., “corrupt”, “salvage”).
CS	The word is an intra-code switched word, combining both Filipino and English elements in the same word (e.g., “naglunch”, “pina-explain”).
OTH	The word does not fit into the above categories. This includes punctuation marks and other symbols, hashtags (e.g., “#BringHimHome”), abbreviations (e.g., “DOST”), names, regardless of the origin of the name (e.g., “Pablo”, “Rochelle”, “Luneta”, “Quiapo”), numbers (e.g., “67”, “2:00 AM”), emojis, onomatopoeic words and expressions (“haha”, “grr”), among others.

In the case of misspellings and non-standard spellings, annotate the word based on the intended meaning of the word. For example, “pde” is a shorthand form of “puwede”, so this should be annotated as **FIL**. Likewise, “pagmamahl” is missing an “a” in the middle, but it is clearly intended to be a Filipino word, so it should also be annotated as **FIL**. However, please remember that abbreviations should be annotated as **OTH**, even if they are abbreviations of Filipino or English words. For example, “LOL” is an abbreviation of “laugh out loud”, but it should be annotated as **OTH**, because it is an abbreviation and not just a shortened form of the word.

In the case of words that are supposed to be one word but are separated by a space, please annotate them separately. For example, the word “magpunta” (normally should be **FIL**) may appear as two separate tokens “mag” and “punta” in the dataset. In this case, annotate both of them as **FIL**. On the other hand, if the word “mag switch” (normally should be **CS**) appears as two separate tokens “mag” and “switch”, then annotate the first token as **FIL** and the second token as **ENG**.

2.2.2 Example Annotations

Important: Please pay careful attention to the following examples to ensure a consistent annotation process. When in doubt, you may consult the instructor for clarification.

23 na flood control projects ang na-award sa St OTH Gerrard Construction OTH
OTH FIL ENG ENG ENG FIL CS FIL OTH OTH OTH ENG OTH

- **Note:** “St. Gerrard” are all marked as **OTH** because “St” is an abbreviation, and “Gerrard” is a name. However, “Construction” is marked as **ENG** because it is an English word. Note that if “St” was written as “Saint”, then it should have been marked as **ENG** instead.

Ang Chinito Walkers ay mga sikat na content creators mula sa DLSU
FIL FIL ENG FIL FIL FIL FIL ENG ENG FIL FIL OTH OTH

- **Note:** “Chinito Walkers” can be viewed as a proper noun, but since it is not a name, it should be marked as **FIL** and **ENG** respectively. “DLSU” is marked as **OTH** because it is an abbreviation, even though its meaning contains an English word (i.e., University).

Madami ang nag-march sa 13 trillion peso march sa EDSA monument at Luneta Park OTH
FIL FIL CS FIL OTH ENG ENG ENG FIL OTH ENG FIL ENG OTH

- **Note:** “nag-march” is marked as **CS** because it is an intra-code switched word, combining the Filipino affix “nag-” with the English root word “march”. “EDSA” and “Luneta” are marked as **OTH** because they are names, even though they are also well-known places in the Philippines. However, “Park” is marked as **ENG** because it is an English word, even though it is used in the context of Filipino place names.

2.3 PinoyBot Implementation

You are provided a skeleton Python file named `pinoybot.py`. This contains a function called `tag_language`, which you must complete for this project. The function accepts a single Filipino passage, and is expected to automatically tag each token in the language with the appropriate tag. We will only have four classes for this project: **ENG** for English, **FIL** for Filipino, **CS** for intra-code switched, and **OTH** for Other.

The parameter for the function is a list of strings, which represent the tokens. For example, the sentence Love Kita
ENG FIL OTH
will be passed to the function as `['Love', 'kita', '.']`, containing 3 tokens. The function is expected to return another list, containing the predicted tags for each token. In this case, if the predictions are correct, the function is expected to return `['ENG', 'FIL', 'OTH']`.

2.3.1 Model Training and Evaluation

To predict the language of each word, you are expected to train a supervised machine learning model (e.g., decision tree, Naïve Bayes) that takes a list of features extracted from a word, and then outputs a language tag for that word (either **FIL**, **ENG**, or **OTH**). To train the model, you can use the `scikit-learn` library, a popular machine learning library in Python, which can handle the training and evaluation process for you.

It is recommended that you go through a few tutorials showing how to use the `sklearn` library to gain a good understanding of how to format the input data, instantiate and train the model, make predictions, and interpret the results:

- [Decision Tree reference for sklearn](#)
- [Naïve Bayes reference for sklearn](#)

You must identify a set of relevant features (i.e., feature engineering) for identifying the language of a given word. Some ideas you may want to consider are:

- presence of letters (number of a's, ratio of vowels, etc.)
- capitalization (is first letter capital, ratio of capital letters, etc.)
- arrangement of letters (character-level n-grams, etc.)
- properties of words surrounding it in the sentence (previous word is predicted as English, previous word ends in a vowel, etc.)

To train the model, you must convert the annotated text samples into a feature matrix and label list, such as the one shown below:

Word (not included as feature)	Feature 1	Feature 2	Feature 3	...	Label
Word 1	value	value	value	...	ENG
Word 2	value	value	value	...	FIL
Word 3	value	value	value	...	OTH
...	...	...	...	...	...

This will serve as the training data for your classifier model/s.

You must observe a 70-15-15 train-validation-test split in training and evaluating the model. It is up to you implement this split appropriately. You are expected to test your classifier model and report its performance on the set.

2.3.2 General Flow for PinoyBot

You must train the test the classifier model separately from the `tag_language` function. Once trained, you can save the model so that it can be called in the `tag_language` model to make predictions. The general flow for the `tag_language` function is as follows:

1. Input: list of strings, representing the tokens in the passage
2. Convert the list of tokens into the feature matrix by extracting the relevant features
3. Call the trained model to predict the tag for each token, assign the results to a new list
4. Return the list of tags

Please make sure that your function is returned in the correct format. The length of the list returned should be equal to the number of tokens in the input, and each tag can only be either **ENG**, **FIL**, or **OTH**. Failure to comply with this may cause problems with the automated checking process.

Please observe the following restrictions:

- You are not allowed to use English or Tagalog dictionaries to directly infer the language of a word (e.g., searching a list of words in a Filipino dictionary and if a specific word appears there, then it must be a Filipino word). While this approach can be helpful in many cases (but not all), we want you to focus your efforts on training the model based on the features of the word itself, rather than using existing repositories. You are allowed to use dictionaries and language resources to help you identify useful features, but you are not supposed to directly look up the language of a word using these resources.
- You are not allowed to use any other existing language identification models, as this would defeat the purpose of training your own.

3 Report

In addition to the PinoyBot codes, you are required to write a report documenting the implementation of the project. The report should contain the following:

1. **Classifier Model:** A section describing what classifier model was used, the hyperparameters, and the list of features used. Explain briefly what influenced the final list of features and the final configuration of the model.
2. **Evaluation and Performance:** A section containing an empirical evaluation of the classifier model on the test set, using relevant metrics for classification such as accuracy, precision, recall, and F1-score. It should also mention how the test set was split from the training data. Finally, a concise analysis of the results, including the strengths and weakness of the classifier should be included.
3. **Challenges:** A discussion on the difficulties encountered in developing the bot. What are the challenges that make it difficult for computers to automatically perform language identification? What are some future directions we can explore to address these challenges?
4. **Table of Contributions:** A table showing the contributions of each member to the project.

The report should **not** exceed four pages with A-4 sized paper. If there is a strong need to have more pages, the group must ask permission from the instructor and justify it. There is no restriction on the formatting, as long as it is readable. Please be concise in writing the report and avoid repeating already-known definitions. The bulk of the report should contain the group's personal ideas, insights, and implementations. Minimize the reiteration of already-known definitions. **You are not allowed to use generative AI for the report, even for writing improvements.** The report should be an accurate reflection of the group's personal understanding and engagement with the project.

4 Deliverables

There are two deliverables for this project: a zip file containing the source files for the project, and a pdf file containing the report. The zip file should contain the following: (1) all the source files needed to run the bot (including `pinoybot.py` which contains the `tag_language` function, as well as the trained model file), (2) the codes used for training the model (this can be placed in a separate folder). There is no need to include the dataset that was used to train the model, but **please be ready to present it upon the request of the instructor.**

5 Academic Honesty

Honesty policy applies. Please take note that you are **NOT** allowed to borrow and/or copy-and-paste in full or in part any existing related program code from the internet or other sources (such as printed materials like books, or source codes by other people that are not online). **Violating this policy is a serious offense in De La Salle University and will result in a grade of 0.0 for the whole course.**

Please remember that the point of this project is for all members to learn something and increase their appreciation of the concepts covered in class. Each member is expected to be able to explain the different aspects of their submitted work, whether part of their contributions or not. **Failure to do this will be interpreted as a failure of the learning goals, and will result in a grade of 0 for that member.**

6 Data Usage

Please note that your contributions in validating the annotations of the datasets can be used in future research to advance the field of NLP in the Philippines.